

MetaForge Recruiter Application V2

Complete Database Design & Relationship Guide

Purpose: A single reference to understand the database structure, collections, field data types, relationships, security boundaries, indexes, and end-to-end recruiter workflow.

Database: MongoDB with Mongoose **Backend:** NestJS **Architecture:** Multi-tenant RBAC

1. Database at a Glance

The supplied master architecture defines **18 core MongoDB collections** across eight functional domains. The implementation reports from the V2 backend verification later reported **13 collections** in the current database state. These figures describe different things: 18 is the master design catalog; 13 is the latest implementation verification count. The document keeps them separate so the design is not confused with the currently observed database state.

Metric	Design / Verified Information
Database	MongoDB v7+ / Mongoose v8+
Core collections in master design	18
Latest reported V2 DB count	19,545 documents across 13 collections
Tenant key	orgId (ObjectId)
Primary identifier	MongoDB _id (ObjectId)
Public/business IDs	Examples: REQ-XXX, CAND-XXX, SUB-XXX, INT-XXX, OFR-XXX, TM-XXX
Access model	RBAC + orgId/data scoping
Deletion model	Soft delete via deletedAt on primary business collections
Audit model	Entity history + audit logs

2. The 18 Core Collections

#	Collection	Domain	Purpose
1	organizations	Tenant Context	Organization accounts and global settings
2	users	Identity & Access	User accounts, credentials and role assignments
3	roles_permissions	Identity & Access	RBAC permission matrix
4	teams	Identity & Access	Recruiter / Team Lead groupings
5	clients	Client Management	B2B clients, POCs and SLA rules
6	requirements	Demand Lifecycle	Job demands, openings, budgets and assignees
7	requirement_histories	Demand Lifecycle	Requirement status/reassignment history
8	candidates	Talent Pool	Candidate profiles, skills, CTC and experience
9	candidate_documents	Talent Pool	Resume/identity document mappings
10	submissions	Submission Workflow	Candidate-to-requirement submission state
11	submission_histories	Submission Workflow	Submission stage progression history
12	interviews	Interview Engine	Interview rounds and schedules
13	interview_feedbacks	Interview Engine	Interview ratings, feedback and rejection notes
14	offers	Offer & Onboarding	Offers, CTC and joining status
15	notifications	Notifications	User in-app notification feed
16	ai_parsing_jobs	AI Engine	Async resume/JD extraction queue logs
17	ai_match_scores	AI Engine	Persistent JD-candidate match scores

#	Collection	Domain	Purpose
18	audit_logs	System Audit	Immutable security/activity logs

3. Relationship Map — How the Tables Are Connected

MongoDB does not use relational SQL foreign keys in the traditional sense. The design uses **ObjectId references** between collections. These references create logical relationships while keeping documents in separate collections.

Parent / Source	Child / Target	Reference	Relationship
organizations	users	users.orgId → organizations._id	1 → many
organizations	clients	clients.orgId → organizations._id	1 → many
organizations	requirements	requirements.orgId → organizations._id	1 → many
organizations	candidates	candidates.orgId → organizations._id	1 → many
roles_permissions	users	users.role/roleCode → role permission definition	1 → many
teams	users	team membership	1 → many / grouping
clients	requirements	requirements.clientId → clients._id	1 → many
users	requirements	assignedLeadId / assignedRecruiterIds	1 → many
requirements	submissions	submissions.requirementId → requirements._id	1 → many
candidates	submissions	submissions.candidateId → candidates._id	1 → many
users	submissions	submissions.recruiterId → users._id	1 → many
submissions	interviews	interviews.submissionId → submissions._id	1 → many
submissions	offers	offers.submissionId → submissions._id	1 → 0/1
candidates	candidate_documents	candidate_documents.candidateId → candidates._id	1 → many
file_metadata	candidate_documents	candidate_documents.fileId → file metadata	1 → 0/1+
interviews	interview_feedbacks	feedback.interviewId → interviews._id	1 → many
users	notifications	notifications.userId → users._id	1 → many
users	audit_logs	audit_logs.userId → users._id	1 → many

4. Core Business Relationship Chain

Organization → Client → Requirement → Submission ← Candidate → Interview → Offer

A requirement represents a job demand. A candidate belongs to the organization and can be submitted against many requirements. A submission is the central transaction connecting a candidate to a requirement. Interviews and offers then progress from that submission. This makes **submissions** the main pipeline/transaction collection.

5. Common Data Types

Type	Meaning in this design	Examples
ObjectId	MongoDB document/reference identifier	_id, orgId, clientId, candidateId
String	Text, codes, names, URLs, statuses	name, reqCode, status, email
Number	Numeric values and counters	openings, CTC, experienceYears
Date	Timestamp/date fields	createdAt, joiningDate, deletedAt
[String]	Array of strings	skills, primarySkills, preferredLocations
[ObjectId]	Array of MongoDB references	assignedRecruiterIds
Boolean	True/false flags where applicable	isActive, revokeRequested
Null / optional	Unset state for fields such as read/deleted timestamps	readAt, deletedAt

6. Field-Level Data Dictionary — Confirmed Core Schemas

The supplied master document gives complete field-level definitions for six core collections. The remaining collection names/purposes and their relationships are defined, while their full field-by-field definitions are referenced in the repository data dictionary but are not included in the supplied document. They are therefore not invented here.

6.1 `requirements`

Field	Type	Purpose
orgId	ObjectId	Tenant reference
reqCode	String	Unique requirement code
clientId	ObjectId	Client reference
clientName	String	Denormalized client name
title	String	Job title
jobDomain	String	Domain category
openings	Number	Required headcount
filledOpenings	Number	Placed/joined headcount
status	String	Open / In Progress / On Hold / Fulfilled / Cancelled
priority	String	P1-Critical / P2-High / P3-Medium / P4-Low
experienceMinYears	Number	Minimum experience
experienceMaxYears	Number	Maximum experience
budgetMinLpa	Number	Minimum CTC
budgetMaxLpa	Number	Maximum CTC
location	String	Job location
primarySkills	[String]	Required skills
secondarySkills	[String]	Preferred skills
description	String	Full JD
assignedLeadId	ObjectId	Team Lead reference
assignedRecruiterIds	[ObjectId]	Recruiter references
emailArrivedTime	Date	Email ingestion time
slaDays	Number	SLA target
deletedAt	Date	Soft delete timestamp
schemaVersion	Number	Schema revision

6.2 `candidates`

Field	Type	Purpose
orgId	ObjectId	Tenant reference
candidateId	String	Candidate public ID
name	String	Candidate full name
email	String	Unique candidate email
phone	String	Unique candidate phone
currentCompany	String	Current employer
currentDesignation	String	Current role
experienceYears	Number	Total experience

Field	Type	Purpose
currentCtcLpa	Number	Current CTC
expectedCtcLpa	Number	Expected CTC
noticePeriodDays	Number	Notice period
location	String	Current location
preferredLocations	[String]	Target locations
skills	[String]	Technical skills
education	String	Highest education
resumeFileId	ObjectId	Resume/file reference
createdBy	ObjectId	Recruiter who created record
deletedAt	Date	Soft delete timestamp
schemaVersion	Number	Schema revision

6.3 `submissions`

Field	Type	Purpose
orgId	ObjectId	Tenant reference
submissionId	String	Submission public ID
requirementId	ObjectId	Requirement reference
candidateId	ObjectId	Candidate reference
recruiterId	ObjectId	Submitting recruiter
stage	String	Pipeline stage
submittedAt	Date	Submission timestamp
leadApprovedAt	Date	Lead approval time
leadRejectedAt	Date	Lead rejection time
leadRejectionReason	String	Rejection reason
clientForwardedAt	Date	Client-forward time
overallFeedback	String	Summary feedback
deletedAt	Date	Soft delete timestamp
schemaVersion	Number	Schema revision

6.4 `interviews`

Field	Type	Purpose
orgId	ObjectId	Tenant reference
interviewId	String	Interview public ID
submissionId	ObjectId	Submission reference
candidateId	ObjectId	Candidate reference
requirementId	ObjectId	Requirement reference
round	String	Interview round
dateTime	Date	Scheduled date/time
durationMinutes	Number	Duration
interviewerName	String	Evaluator name

Field	Type	Purpose
interviewerEmail	String	Evaluator email
meetingLink	String	Meeting URL
status	String	Scheduled / Completed / Cancelled / Rescheduled / No Show
deletedAt	Date	Soft delete timestamp
schemaVersion	Number	Schema revision

6.5 `offers`

Field	Type	Purpose
orgId	ObjectId	Tenant reference
offerId	String	Offer public ID
submissionId	ObjectId	Unique submission reference
candidateId	ObjectId	Candidate reference
requirementId	ObjectId	Requirement reference
offeredCtc	Number	Annual offered CTC
offerReleaseDate	Date	Offer issue date
joiningDate	Date	Target joining date
status	String	Offer Released / Accepted / Joined / Declined / Backed Out
declinedReason	String	Decline reason
notJoinedNote	String	Back-out notes
deletedAt	Date	Soft delete timestamp
schemaVersion	Number	Schema revision

6.6 `notifications`

Field	Type	Purpose
orgId	ObjectId	Tenant reference
userId	ObjectId	Recipient user
type	String	Notification type
title	String	Alert heading
message	String	Alert body
entityId	ObjectId	Linked entity
entityType	String	submission / interview / offer / requirement
readAt	Date	Read timestamp; null when unread
schemaVersion	Number	Schema revision

7. Collection Details Not Fully Expanded in the Supplied Master Document

The master catalog identifies these collections and their roles, but the supplied file does not provide their complete field-level schemas. The repository points to `09C\_Data\_Dictionary.md` as the field-by-field source. These should be used as the authoritative field reference before implementation changes are made.

Collection	Purpose	Schema status	
organizations	Tenant account/global settings	Field-level schema not included in supplied	master file
users	Credentials, roles, assignments	Field-level schema not included in supplied	master file
roles_permissions	RBAC permission matrix	Field-level schema not included in supplied	master file
teams	Team/group membership	Field-level schema not included in supplied	master file
clients	Client accounts, POCs, SLA rules	Field-level schema not included in supplied	master file
requirement_histories	Requirement status/reassignment history	Field-level schema not included in supplied	master file
candidate_documents	Resume/document mappings	Field-level schema not included in supplied	master file
submission_histories	Submission stage history	Field-level schema not included in supplied	master file
interview_feedbacks	Interview ratings/feedback	Field-level schema not included in supplied	master file
ai_parsing_jobs	Async parsing queue logs	Field-level schema not included in supplied	master file
ai_match_scores	JD-candidate match scores	Field-level schema not included in supplied	master file
audit_logs	Immutable activity/security logs	Field-level schema not included in supplied	master file

8. Security & Tenant Isolation

- **orgId is the tenant boundary:** operational documents carry an indexed orgId reference.
- **superadmin:** global access across tenant organizations and system settings/logs.
- **admin:** full access within the assigned organization.
- **lead:** requirement access based on assigned lead/team recruiter scope.
- **recruiter:** requirement/submission access based on recruiter assignment.
- **devteam:** read-only diagnostics/system health/history access.
- **JWT:** protected API routes require authenticated bearer tokens.
- **Soft delete:** primary business collections use deletedAt; active queries exclude deleted records.

9. Indexes & Performance

Collection	Index / Rule
requirements	{ orgId: 1, status: 1, priority: 1, createdAt: -1 }
candidates	{ orgId: 1, email: 1 } unique; { orgId: 1, phone: 1 } unique
candidates	Text index on name, currentCompany, skills
submissions	{ orgId: 1, requirementId: 1, stage: 1, createdAt: -1 }
submissions	Unique guard: { orgId: 1, candidateId: 1, requirementId: 1, deletedAt: 1 }
offers	{ orgId: 1, submissionId: 1, deletedAt: 1 } unique
notifications	{ orgId: 1, userId: 1, readAt: 1, createdAt: -1 }
notifications	TTL on createdAt: 7,776,000 seconds / 90 days

10. Data Integrity Rules

- Candidate cannot be submitted to the same requirement twice because of the submissions compound unique constraint.
- Candidate email and phone are unique per organization.
- Creating an Offer moves the related submission stage to **Offered**.
- Updating an Offer to **Joined** moves the submission to **Placed** and increments requirements.filledOpenings.
- Primary business collections use soft deletion rather than destructive deletion.
- Requirement actions create history records for CREATE, UPDATE, ASSIGN, REVOKE and DELETE.
- Submission stages are tracked through submission history.

11. How the Database Supports the Recruiter Workflow

Business step	Main collections involved	What is stored
1. Organization/User setup	organizations, users, roles_permissions, teams	Tenant, users, roles and team assignments
2. Client setup	clients	Client, contacts and SLA information
3. Requirement creation	requirements, requirement_histories	Job demand, openings, budget, skills, assignment and history
4. Candidate pool	candidates, candidate_documents, file metadata	Candidate profile, resume/document references
5. Candidate submission	submissions, submission_histories	Candidate-to-requirement pipeline state
6. Interview	interviews, interview_feedbacks	Interview schedule, round, result and feedback
7. Offer	offers, submissions, requirements	Offer details, joining status and placement progression
8. Notifications	notifications	User alerts for workflow events
9. AI	ai_parsing_jobs, ai_match_scores	Resume/JD parsing jobs and candidate match scores
10. Audit	audit_logs	Security and activity history

12. Auto Mail Extractor & Resume Parsing — Where They Fit

From the project workflow discussed in the V2 work, Auto Mail Extractor and Resume Parsing are V1 capabilities that still need to be audited and adapted into V2. The master database design already reserves AI-related collections such as `ai\_parsing\_jobs` and `ai\_match\_scores`, and the requirement/candidate models contain the target business data. The exact V1 implementation details, parser fields and ingestion mechanism are not contained in the supplied master database document, so those details should be taken from the V1 code before migration.

Target logical flow: Requirement Email → Auto Mail Extractor → requirements → Recruiter Workspace → Resume/Document → Resume Parsing → candidates → AI Matching → submissions.

13. Implementation Status From Our V2 Work

Area	Status
MongoDB migration/integration	Completed and verified
Authentication	Completed and verified
Users & Roles	Completed and verified
Requirements	Completed and verified
Submissions & Workspace	Completed and verified
Candidates	Completed and verified
Interviews	Completed and verified
Clients	Completed and verified

Area	Status
Analytics	Completed and verified
Audit Logs	Completed and verified
Teams	Completed and verified
Files	Completed and verified
AI Gateway	Completed and verified
Candidate Blacklist	Completed and verified
Saved Searches	Completed and verified
Email Templates	Completed and verified
Offers & Notifications	Verification still required in our workflow
Auto Mail Extractor	V1 audit/adaptation still required
Resume Parsing	V1 audit/adaptation still required
Frontend ↔ Backend integration	Next major phase

14. What You Should Understand From This Document

- The database is organized around a tenant (`organizations`) and user/role security.
- `requirements` is the demand side of the system; `candidates` is the talent side.
- `submissions` is the central bridge between a candidate and a requirement.
- `interviews` and `offers` progress a submission toward placement.
- History/audit collections preserve what happened over time instead of overwriting the complete historical trail.
- AI collections support asynchronous parsing and persistent matching results.
- Notifications provide user-facing workflow alerts.
- ObjectId fields are the main mechanism used to connect documents between collections.
- Public IDs such as REQ-XXX and CAND-XXX make business records easier to reference while MongoDB `\_id` remains the underlying document identifier.
- Indexes enforce uniqueness and support efficient tenant-scoped queries.

15. Source & Authority Notes

This guide is based primarily on the supplied **Master Database Design Document** and the V2 implementation/test information established in this conversation. The master document identifies the 18-collection architecture and explicitly references the repository files `09\_FINAL\_HIGH\_LEVEL\_DATABASE\_ARCHITECTURE.md`, `09C\_Data\_Dictionary.md`, `09D\_Index\_Strategy.md`, and `09E\_Audit\_And\_History.md` as the deeper database references. Where those detailed files were not supplied, this guide does not invent field definitions.